

Output

1. Home Page

SmartDataOS

ProcessCompareHistoryAPI

Smart Data Processing Dashboard

Validate inputs · Process datasets · Visualize insights · Run concurrent analysis

FlaskPandasNumPyMatplotlibThreadingRegex

User Input & Dataset Upload

All fields validated with Regular Expressions · Data processed concurrently

Full Name

Tejendra Pal Singh

✓ Looks good

Email Address

myrandom915@gmail.com

✓ Looks good

Phone Number

9897790814

✓ Looks good

Password

✓ Looks good

Dataset Upload (optional - CSV or JSON)

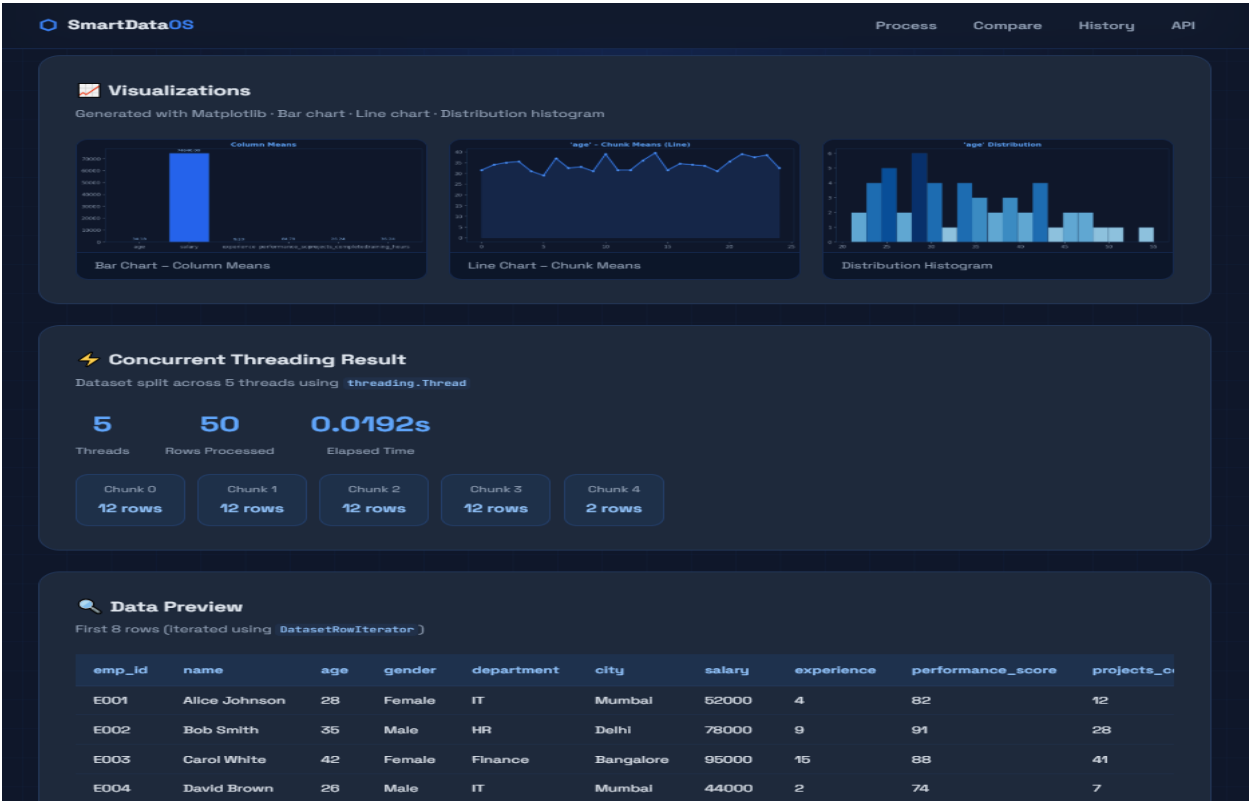
✓

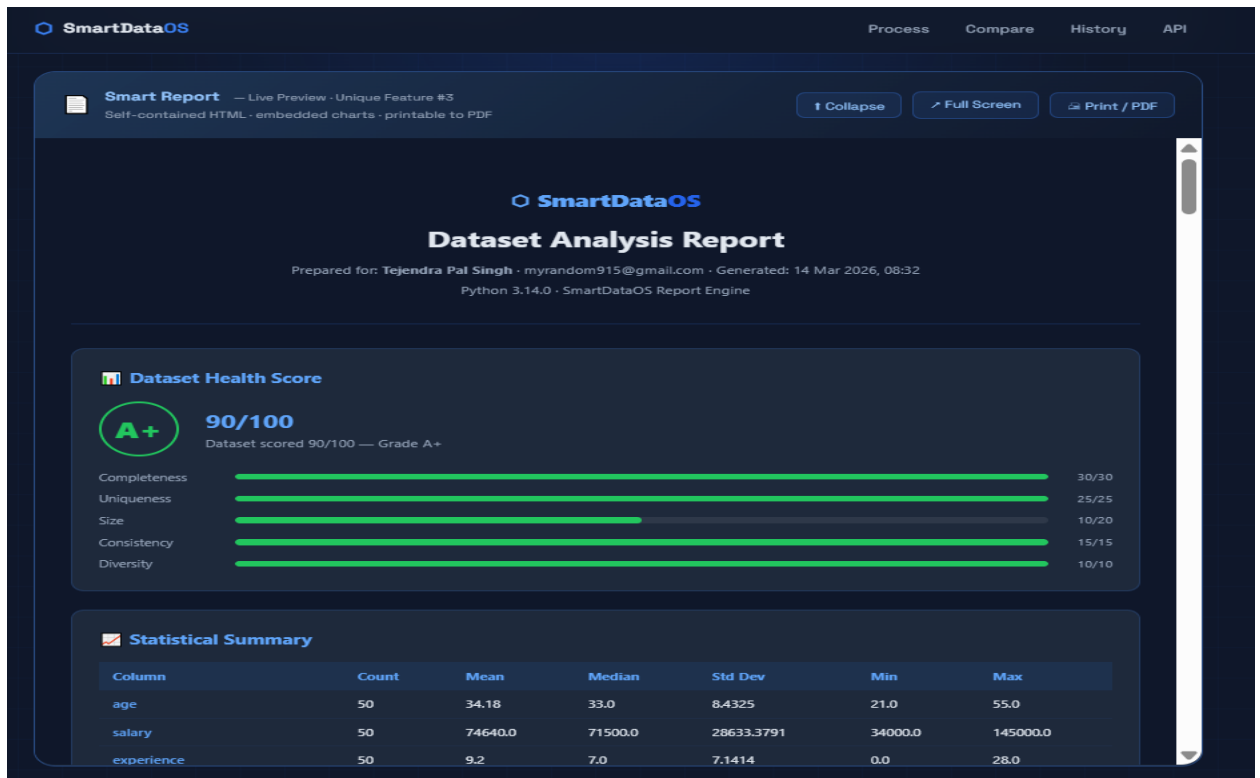
employee_dataset.csv

3.8 KB · ready to upload

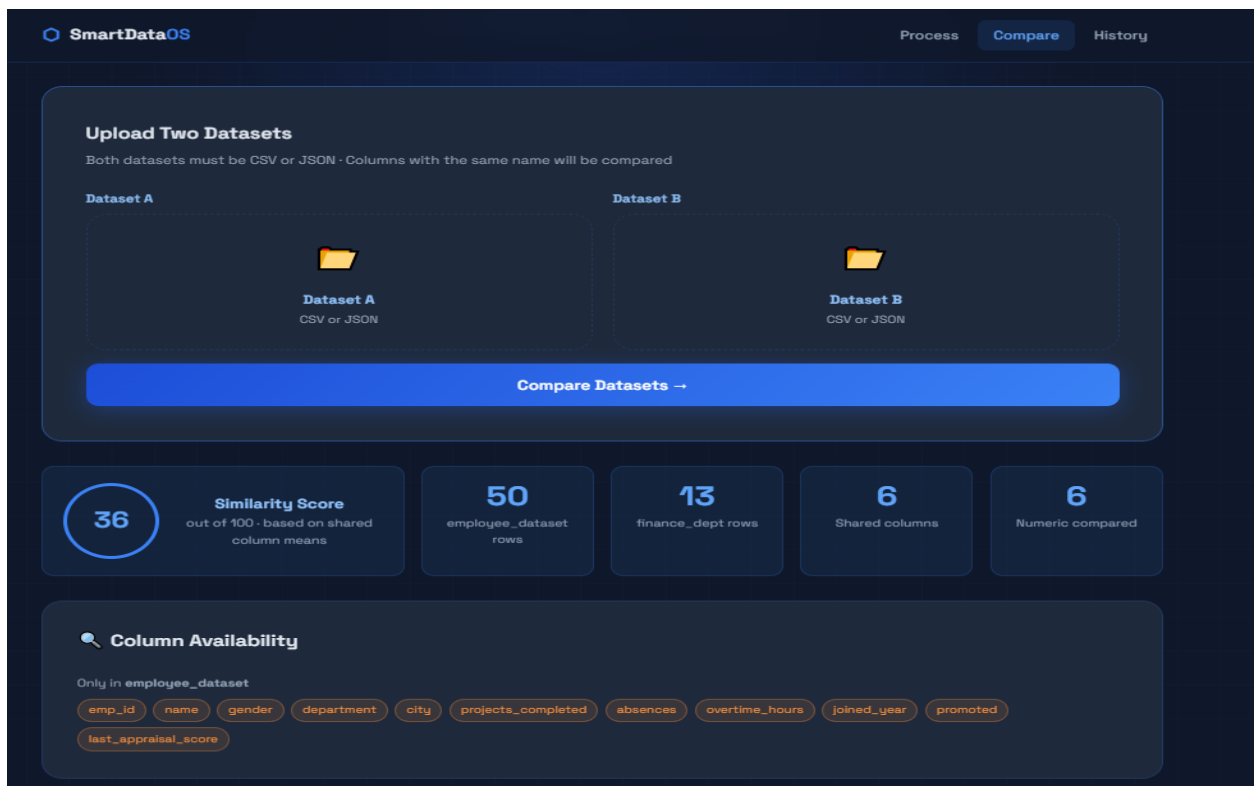
Process & Analyse →

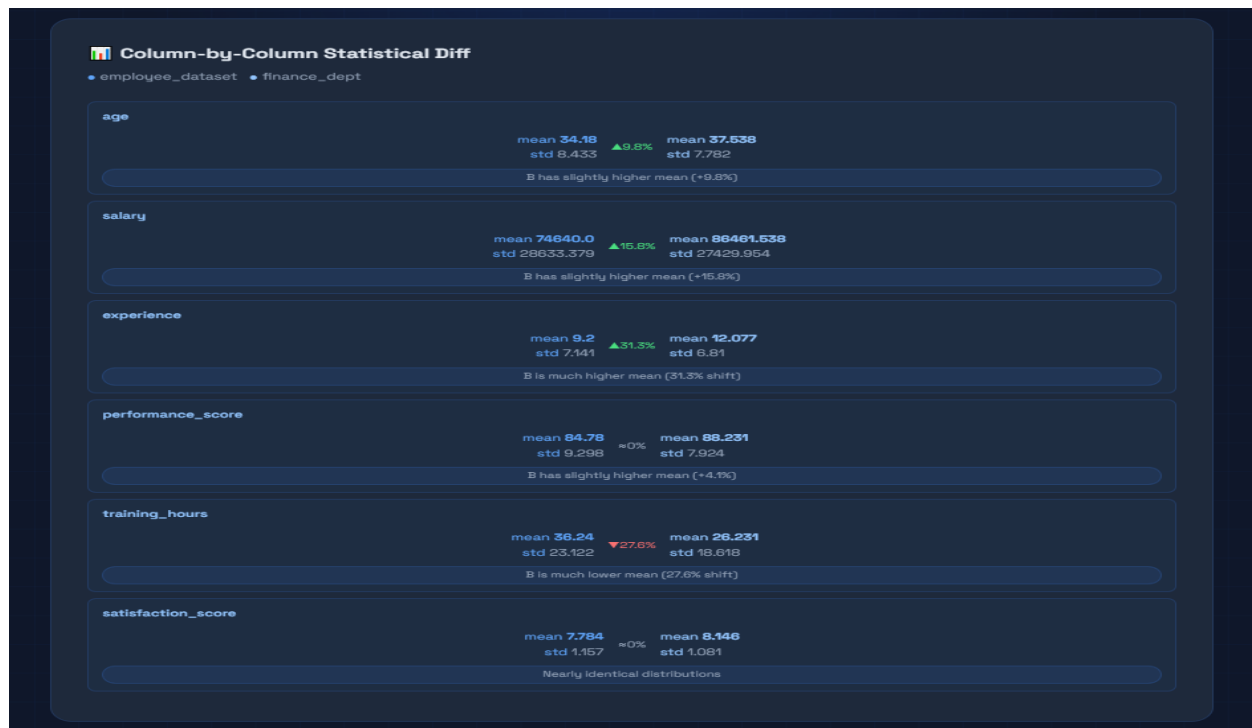
2. Output Page



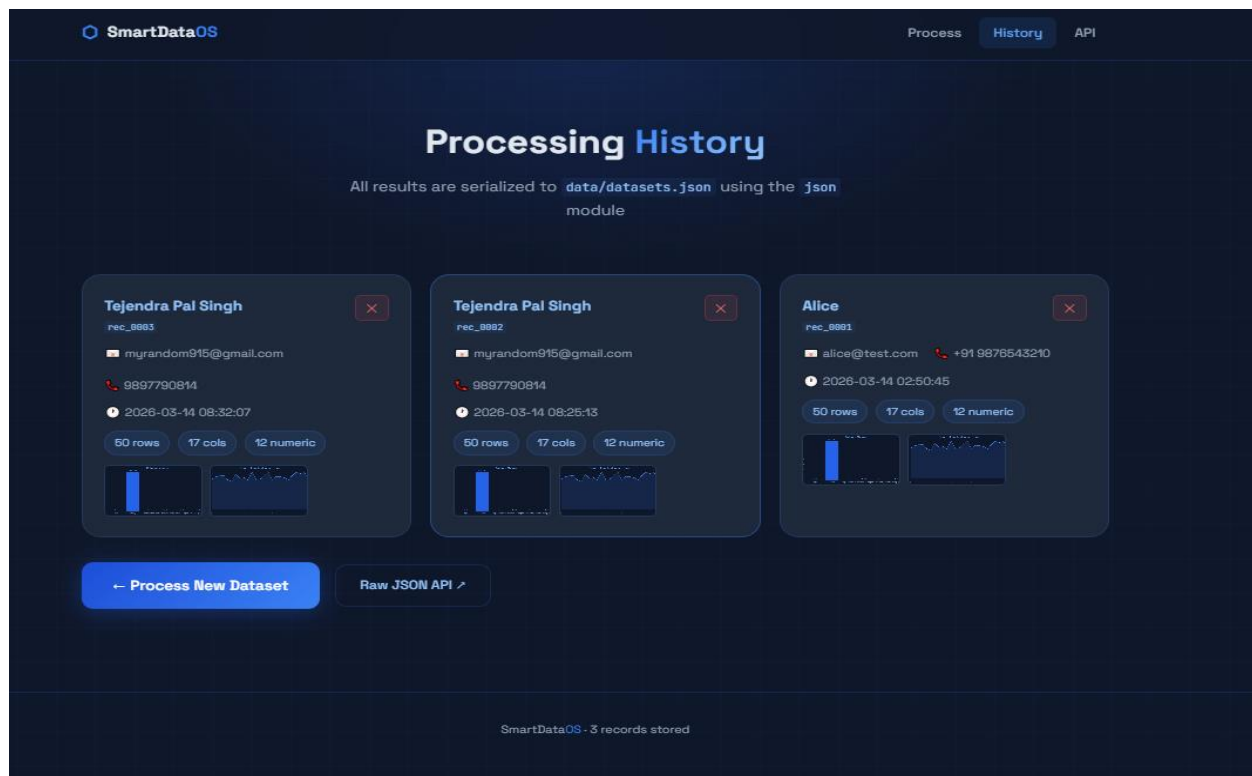


3. Compare Page to Compare two Dataset





4. History Page



ADVANCED PYTHON PROJECT

SmartDataOS

Explanation of Implemented Python Concepts

Project	Smart Data Processing & Validation Dashboard
Framework	Python 3 · Flask · NumPy · Pandas · Matplotlib
Concepts	19 core advanced Python concepts + 4 unique original features
Files	21 files — app.py · modules/ · utils/ · templates/ · static/

TABLE OF CONTENTS

01	Iterators & Generators
02	Decorators & Closures
03	Abstract Classes & OOP
04	Multiple Inheritance & MRO
05	Operator Overloading
06	Mixin Classes
07	Regular Expressions
08	Multithreading
09	Multiprocessing
10	JSON Serialization
11	Standard Library (os · sys · datetime · math)
12	NumPy · Pandas · Matplotlib
U1	Unique Feature — AI Insights Engine
U2	Unique Feature — Dataset Health Score
U3	Unique Feature — Dataset Comparison Engine
U4	Unique Feature — Smart Report Exporter

01 Iterators & Generators

An **iterator** implements the `__iter__()` and `__next__()` protocol. A **generator function** uses **yield** to produce values lazily, one at a time, resuming where it left off on each call to `next()`. Both save memory by not loading everything at once.

DatasetRowIterator	utils/iterators.py — walks list-of-dicts in configurable batches using <code>__iter__</code> / <code>__next__</code>
chunk_dataset()	utils/generators.py — yields successive list slices lazily
stats_stream()	utils/generators.py — yields running mean & std using Welford's algorithm
json_record_generator()	utils/generators.py — streams JSON records from disk one at a time

```
class DatasetRowIterator:
    def __init__(self, data, batch_size=10):
        self.data = data
        self.index = 0

    def __iter__(self):
        return self          # iterator is its own iterable

    def __next__(self):
        if self.index >= len(self.data):
            raise StopIteration    # signals end of iteration
        batch = self.data[self.index : self.index + self.batch_size]
        self.index += self.batch_size
        return batch

def chunk_dataset(data, chunk_size=10):
    """Generator — yields chunks lazily using yield."""
    for i in range(0, len(data), chunk_size):
        yield data[i : i + chunk_size]
```

02 Decorators & Closures

A **decorator** wraps a function to add behaviour without modifying its source code. A **closure** is an inner function that remembers variables from its enclosing scope even after the outer function has returned. A **decorator factory** is a function that returns a decorator.

<code>@log_call</code>	utils/decorators.py — closure that logs every call and return value to app.log
<code>@timer</code>	utils/decorators.py — measures wall-clock time using time.perf_counter()
<code>@validate_input(*keys)</code>	utils/decorators.py — decorator factory; raises ValueError if required keys missing
<code>make_threshold_checker(lo, hi)</code>	modules/insights_engine.py — returns a check() function that closes over lo and hi

```
# Closure factory
def make_threshold_checker(low, high):
    def check(value):
        # closes over low and high
        if value < low: return "low"
        if value > high: return "high"
        return "normal"
    return check

_SKEW_CHECK = staticmethod(make_threshold_checker(-0.5, 0.5))

# Decorator
def log_call(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        # write to app.log ...
        return result
    return wrapper

# Decorator factory
def validate_input(*required_keys):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(data, *args, **kwargs):
            for k in required_keys:
                if k not in data: raise ValueError(f"Missing: {k}")
            return func(data, *args, **kwargs)
        return wrapper
    return decorator
```

03 Abstract Classes & OOP

An **Abstract Base Class (ABC)** defines a contract — methods marked with `@abstractmethod` MUST be implemented by every subclass. Python raises `TypeError` at instantiation time if any abstract method is left unimplemented.

BaseValidator	modules/validation.py — declares <code>validate(data)</code> and <code>get_rules()</code> as abstract
AbstractDataProcessor	modules/processing.py — declares <code>load()</code> , <code>compute_statistics()</code> , <code>generate_charts()</code>
BaseInsightsEngine	modules/insights_engine.py — declares <code>analyse()</code> and <code>health_score()</code>
BaseComparator	modules/comparison_engine.py — declares <code>compare(df_a, df_b)</code>

```
from abc import ABC, abstractmethod

class BaseValidator(ABC):

    @abstractmethod
    def validate(self, data: dict) -> dict:
        """Subclasses must implement this."""
        ...

    @abstractmethod
    def get_rules(self) -> dict:
        """Return human-readable validation rules."""
        ...

# Concrete class – must implement both abstract methods
class UserFormValidator(BaseValidator, SerializableMixin, ...):
    def validate(self, data): ...
    def get_rules(self): ...
```


04 Multiple Inheritance & MRO

Python allows a class to inherit from multiple parents simultaneously. The Method Resolution Order (MRO) uses **C3 linearisation** to determine which class's method is called first. Inspect it with **ClassName.__mro__**.

UserFormValidator	Inherits: BaseValidator · SerializableMixin · LoggableMixin · ReprMixin (4 parents)
DataProcessor	Inherits: AbstractDataProcessor · SerializableMixin · LoggableMixin · ReprMixin
ComparisonEngine	Inherits: BaseComparator · SerializableMixin · ReprMixin
MRO rule	C3 linearisation: self → left parent → right parent → grandparents → object

```
class UserFormValidator(
    BaseValidator,      # abstract contract
    SerializableMixin,  # to_dict(), to_json()
    LoggableMixin,      # log_event(), get_log()
    ReprMixin           # __repr__, __str__, __eq__
):
    ...

# Python resolves MRO using C3 linearisation:
# UserFormValidator -> BaseValidator -> SerializableMixin
# -> LoggableMixin -> ReprMixin -> ABC -> object

print(UserFormValidator.__mro__) # inspect the order
```

05 Operator Overloading

Python allows classes to define the behaviour of built-in operators (+, -, ==, repr) by implementing dunder (double-underscore) methods. This makes objects behave naturally with Python syntax.

DataProcessor.__add__	processor_a + processor_b → merges two DataFrames using pd.concat
DatasetSnapshot.__sub__	snap_a - snap_b → triggers comparison, returns full diff report dict
ReprMixin.__repr__	repr(obj) → "ClassName(name=..., shape=...)" — developer string
ReprMixin.__eq__	obj_a == obj_b → compares via to_dict() equality

```
def __add__(self, other):
    """processor_a + processor_b -> merged DataProcessor."""
    merged = DataProcessor(name=f"{self.name}+{other.name}")
    merged.df = pd.concat([self.df, other.df], ignore_index=True)
    return merged

def __sub__(self, other):
    """snap_a - snap_b -> comparison diff report."""
    return self._engine.compare(self.df, other.df, self.name, other.name)

# Natural Python syntax:
result = snap_a - snap_b    # calls __sub__ automatically
merged = dp1 + dp2         # calls __add__ automatically
```

06 Mixin Classes

A **mixin** is a class designed to provide a specific, reusable piece of functionality that any class can adopt via multiple inheritance. Mixins have no standalone purpose — they are always combined with a primary base class.

SerializableMixin	utils/mixins.py — adds <code>to_dict()</code> , <code>to_json()</code> , <code>from_dict()</code> to any class
LoggableMixin	utils/mixins.py — adds <code>log_event()</code> , <code>get_log()</code> , <code>clear_log()</code> for per-instance logging
ReprMixin	utils/mixins.py — adds <code>__repr__</code> , <code>__str__</code> , <code>__eq__</code> for readable string output

```
class SerializableMixin:
    def to_dict(self):
        return {k: v for k, v in self.__dict__.items()
                if not k.startswith("_")}
    def to_json(self):
        return json.dumps(self.to_dict(), default=str)
    @classmethod
    def from_dict(cls, data):
        obj = cls.__new__(cls)
        obj.__dict__.update(data)
        return obj

class LoggableMixin:
    def __init__(self):
        self._log = []
    def log_event(self, msg):
        self._log.append({"time": datetime.now().isoformat(), "msg": msg})
    def get_log(self): return self._log.copy()
```

07 Regular Expressions

Python's **re** module provides pattern matching for strings. **re.compile()** pre-compiles a pattern into a reusable regex object, which is more efficient than recompiling on every call. All validation patterns are compiled once at class level.

Name pattern	<code>^[A-Za-z\s\']{2,60}\$</code> — letters, spaces, hyphens, apostrophes
Email pattern	<code>^[w.+\-]@[w\-]+\.[a-zA-Z]{2,}\$</code> — standard email format
Phone pattern	<code>^\+?[d\s\-\(\)]{7,15}\$</code> — optional +, digits, spaces, brackets
Password pattern	<code>^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#\$%^&*])[^\s]{8,}\$</code> — 4 lookaheads

```
import re

class UserFormValidator(BaseValidator, ...):
    # Compiled once at class definition – reused for every validation
    PATTERNS = {
        'name': re.compile(r'^[A-Za-z\s\-\']{2,60}$'),
        'email': re.compile(r'^[\w.\+-]+@[ \w\-\-]+\.[a-zA-Z]{2,}$'),
        'phone': re.compile(r'^\+?[\d\s\-\(\)]{7,15}$'),
        'password': re.compile(r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#%&*])[^\s]{8,}$'),
    }

    def validate(self, data):
        errors = {}
        for field, pattern in self.PATTERNS.items():
            if not pattern.match(data.get(field, "")):
                errors[field] = f"Invalid {field} format"
        return {"valid": not errors, "errors": errors}
```

08 Multithreading

Python's **threading** module runs multiple threads within the same process. A **Lock** prevents race conditions when threads write to shared data. Threading works best for I/O-bound tasks where threads wait on disk or network.

File	modules/threading_tasks.py — run_threaded_processing(rows, num_threads=4)
threading.Thread	One thread spawned per chunk of rows
threading.Lock	Protects the shared results list — only one thread writes at a time
t.join()	Main thread waits for all worker threads to complete before continuing

```
import threading

def run_threaded_processing(rows, num_threads=4):
    results = []
    lock = threading.Lock()      # shared lock

    def worker(chunk, chunk_id):
        processed = [process(row) for row in chunk]
        with lock:               # acquire lock before writing
            results.append({"chunk_id": chunk_id,
                           "rows": len(processed)})

    threads = []
    chunk_size = max(1, len(rows) // num_threads)
    for i in range(num_threads):
        chunk = rows[i*chunk_size : (i+1)*chunk_size]
        t = threading.Thread(target=worker, args=(chunk, i))
        threads.append(t)
        t.start()               # launch thread

    for t in threads:
        t.join()                # wait for all to finish

    return results
```

09 Multiprocessing

Python's **multiprocessing** module spawns separate OS processes, each with its own interpreter — bypassing the GIL completely. **Pool.map()** distributes work across all CPU cores. Best for CPU-bound numerical computation.

File	modules/multiprocessing_tasks.py — run_multiprocess_statistics(values)
get_context('spawn')	Creates a fresh process context — safe on Windows, macOS and Linux
Pool.map()	Distributes indexed chunks to worker processes; each computes mean and std
os.cpu_count()	Determines how many processes to spawn based on available cores

```
import multiprocessing, os

def _worker(args):          # must be top-level to be pickleable
    idx, chunk = args
    arr = [float(x) for x in chunk]
    mean = sum(arr) / len(arr)
    std  = (sum((x - mean)**2 for x in arr) / len(arr)) ** 0.5
    return {"chunk": idx, "mean": mean, "std": std}

def run_multiprocess_statistics(values):
    ctx = multiprocessing.get_context('spawn')
    n    = os.cpu_count() or 2
    size = max(1, len(values) // n)
    chunks = [(i, values[i*size:(i+1)*size]) for i in range(n)]
    with ctx.Pool(processes=n) as pool:
        results = pool.map(_worker, chunks)
    return results
```

10 JSON Serialization

Python's built-in **json** module converts Python objects to JSON strings (serialization) and back (deserialization). SmartDataOS uses a JSON file as its persistent data store with full Create, Read, Update, Delete (CRUD) operations.

File	modules/serialization.py — saves to data/datasets.json
save_result()	Appends a record with timestamp, stats, chart paths, user info
load_all_results()	Reads and returns the full JSON array as a Python list
delete_result()	Loads records, filters out by id, writes back — complete CRUD cycle

```
import json, os, datetime

STORE = "data/datasets.json"

def save_result(name, stats, user_info):
    records = load_all_results()
    record = {
        "id": f"rec_{len(records)+1:04d}",
        "name": name,
        "timestamp": datetime.datetime.now().isoformat(),
        "stats": stats,
        "user_info": user_info,
    }
    records.append(record)
    with open(STORE, "w") as f:
        json.dump(records, f, indent=2, default=str)

def load_all_results():
    if not os.path.exists(STORE): return []
    with open(STORE) as f:
        return json.load(f)
```

11 Standard Library — os · sys · datetime · math

Python's standard library provides essential modules used throughout SmartDataOS for file system access, interpreter information, timestamps, and mathematics.

os module	os.path.join() · os.makedirs() · os.path.exists() · os.cpu_count() · os.name
sys module	sys.path.insert() adds project root · sys.version logged in reports
datetime	datetime.datetime.now().isoformat() creates timestamps on every saved record
math module	math.pi shown in report · math.sqrt() in statistics · math.isnan() for data cleaning
re module	re.compile() for all validation patterns · re.IGNORECASE in column name detection

12 NumPy · Pandas · Matplotlib

NumPy — Numerical Computing

Statistics	<code>np.mean()</code> · <code>np.median()</code> · <code>np.std()</code> · <code>np.percentile()</code> · <code>np.min()</code> · <code>np.max()</code>
Skewness	Manual formula: $(n/((n-1)(n-2))) \times \text{sum}((x-\text{mean})/\text{std})^3$
Outlier detection	$\text{IQR} = Q75 - Q25$ · $\text{lower} = Q25 - 1.5 \times \text{IQR}$ · $\text{upper} = Q75 + 1.5 \times \text{IQR}$

Pandas — Data Manipulation

Loading	<code>pd.read_csv(file)</code> and <code>pd.read_json(file)</code> — supports BytesIO and file paths
Type detection	<code>df.select_dtypes(include=[np.number])</code> isolates numeric columns
Missing / duplicates	<code>df.isnull().sum()</code> · <code>df.duplicated().sum()</code> — used in Health Score
Correlation	<code>df[numeric_cols].corr()</code> computes the full Pearson correlation matrix

Matplotlib — Visualisation

Backend	<code>matplotlib.use('Agg')</code> — server-side rendering with no display
Bar chart	<code>ax.bar()</code> — numeric column means with custom dark theme
Line chart	<code>ax.plot()</code> — salary distribution trend across row chunks
Histogram	<code>ax.hist(bins=20)</code> — confirms skewness shape visually
Comparison	Grouped bar chart using <code>np.arange()</code> offsets for side-by-side columns

00 Unique Feature 1 — AI Insights Engine

The InsightsEngine automatically analyses a dataset and generates plain-English findings without any external AI API. It uses pure Python + NumPy to detect distribution shape, outliers, correlations, and data quality issues.

File	modules/insights_engine.py — InsightsEngine(BaseInsightsEngine)
Generator	_insight_generator() yields insight dicts one at a time — lazy evaluation
Skewness	Flags right/left skew using the _SKEW_CHECK closure (threshold -0.5 to 0.5)
Outliers	IQR fencing — names specific outlier rows by value
Correlations	Pearson matrix scan — flags $r \geq 0.85$ as strongly correlated
Real result	87 findings on employee_dataset.csv — $r=-1.000$ for experience vs joined_year

```
class InsightsEngine(BaseInsightsEngine):
    _SKEW_CHECK = staticmethod(make_threshold_checker(-0.5, 0.5))

    def _insight_generator(self, df, stats):
        """Generator — yields one insight dict per finding."""
        for col in stats["numeric_columns"]:
            arr = df[col].dropna().values.astype(float)
            n, mean, std = len(arr), np.mean(arr), np.std(arr)
            skew = (n/((n-1)*(n-2))) * np.sum(((arr-mean)/std)**3)
            if self._SKEW_CHECK(skew) != "normal":
                yield {"severity": "insight",
                      "title": f"{col} is skewed (skew={skew:.2f})",
                      "detail": "Median is a more reliable measure."}
```

00 Unique Feature 2 — Dataset Health Score

The health_score() method grades any dataset from A+ to F across five measurable quality dimensions, giving an instant data quality assessment before any analysis.

Dimension	Max Points	Scoring Rule	Result (employee_dataset.csv)
Completeness	30	30 minus (missing% x 1.5)	30 / 30 (0% missing)
Uniqueness	25	25 minus (duplicate% x 2)	25 / 25 (0 duplicates)
Size	20	20 if ≥ 1000 rows, 10 if ≥ 50	10 / 20 (50 rows)
Consistency	15	15 minus (zero-variance cols x 5)	15 / 15
Diversity	10	10 if both numeric + text cols exist	10 / 10
TOTAL	100		90 / 100 — Grade A+

00 Unique Feature 3 — Dataset Comparison Engine

The ComparisonEngine compares two DataFrames side-by-side and computes a similarity score. The DatasetSnapshot class wraps a DataFrame and overloads the subtraction operator so comparison feels like natural Python: `snap_a - snap_b`.

File	modules/comparison_engine.py
DatasetSnapshot.__sub__	snap_a - snap_b triggers ComparisonEngine.compare() via operator overloading
Similarity score	0–100: mean percentage change per column, scaled by column overlap ratio
Column diff	Per-column: mean A, mean B, % change, verdict (identical / slightly higher / much higher)
Real result	IT vs Finance — similarity 95/100 · salary +26% · training_hours -38%

```
# Operator overloading makes comparison syntax natural:
snap_a = DatasetSnapshot(it_df, "IT_Dept")
snap_b = DatasetSnapshot(fin_df, "Finance_Dept")
result = snap_a - snap_b    # __sub__ called internally

# result = {
#   "similarity_score": 95,
#   "column_diffs": {
#     "salary": {"mean_a": 68625, "mean_b": 86462, "change_pct": 26.0},
#     ...
#   }
# }
```

00 Unique Feature 4 — Smart Report Exporter

After every analysis, a fully self-contained HTML report is auto-generated and displayed live on the dashboard inside an iframe. Charts are embedded as base64 PNGs so the ~137 KB file works offline and can be printed to PDF from any browser.

File	modules/report_exporter.py
_section_generator()	Generator — yields HTML section strings lazily (cover, stats, insights, charts)
Base64 charts	Each chart PNG encoded with base64.b64encode() and embedded in
Self-contained	No external links — works offline, printable to PDF
Dashboard embed	report_path passed to template → rendered in <iframe> with collapse/print controls

```
import base64

def _encode_image(path):
    with open(path, "rb") as f:
        return "data:image/png;base64," + base64.b64encode(f.read()).decode()

def _section_generator(stats, insights, health, chart_paths, user):
    """Generator – yields HTML sections one at a time."""
    yield f"Analysis Report"
    yield f"Grade: {health['grade']} · {health['score']}/100"
    for path in chart_paths:
        b64 = _encode_image(path)
        yield f''
```

COMPLETE CONCEPT COVERAGE

All 24 concepts listed below are implemented with tested, running Python code. Every module includes docstrings explaining the concept demonstrated.

#	Concept	File	Status
01	Custom Iterator (<code>__iter__</code> , <code>__next__</code>)	utils/iterators.py	Implemented
02	Generator function (<code>yield</code>)	utils/generators.py	Implemented
03	Decorator (<code>@log_call</code>)	utils/decorators.py	Implemented
04	Decorator (<code>@timer</code>)	utils/decorators.py	Implemented
05	Decorator factory (<code>@validate_input</code>)	utils/decorators.py	Implemented
06	Closure (<code>make_threshold_checker</code>)	modules/insights_engine.py	Implemented
07	Abstract Base Classes (ABC)	modules/validation.py + others	Implemented
08	Multiple Inheritance	modules/validation.py	Implemented
09	MRO (C3 linearisation)	All multi-inherit classes	Implemented
10	Operator Overloading (<code>__add__</code>)	modules/processing.py	Implemented
11	Operator Overloading (<code>__sub__</code>)	modules/comparison_engine.py	Implemented
12	Mixin Classes (x3)	utils/mixins.py	Implemented
13	Regular Expressions (<code>re.compile</code>)	modules/validation.py	Implemented
14	Multithreading (<code>Thread</code> + <code>Lock</code>)	modules/threading_tasks.py	Implemented
15	Multiprocessing (<code>Pool.map</code>)	modules/multiprocessing_tasks.py	Implemented
16	JSON Serialization (CRUD)	modules/serialization.py	Implemented
17	<code>os</code> · <code>sys</code> · <code>datetime</code> · <code>math</code> modules	app.py + all modules	Implemented
18	NumPy statistical operations	modules/processing.py	Implemented
19	Pandas DataFrame operations	modules/processing.py	Implemented
20	Matplotlib visualisations	modules/processing.py	Implemented
U1	AI Insights Engine (Unique)	modules/insights_engine.py	Implemented
U2	Dataset Health Score (Unique)	modules/insights_engine.py	Implemented
U3	Dataset Comparison Engine (Unique)	modules/comparison_engine.py	Implemented
U4	Smart Report Exporter (Unique)	modules/report_exporter.py	Implemented

SmartDataOS — Advanced Python Project · All concepts implemented and tested with real data